Synthèse de la semaine 2

NumPy, Pandas, nettoyage, visualisation et statistiques : l'essentiel de chaque jour en deux minutes de lecture. Pour chaque jour, les définitions, les résultats à connaître, les pièges qui tombent aux examens et des cas d'examen corrigés.

1. NumPy : calculer sur des tableaux
2. Pandas : explorer un tableau de données
3. Nettoyer des données sales
4. Visualiser pour comprendre
5. Statistiques pour le ML

---

1. NumPy : calculer sur des tableaux DAY 1 · 2 MIN

Définition 1.1 *(ndarray)*

Un `np.array` est un tableau de nombres **de même type** (`dtype`), de forme fixe (`shape`), avec un nombre de dimensions (`ndim`). Une image couleur a la forme `(hauteur, largeur, 3)` .

Proposition 1.2 *(vectorisation et broadcasting)*

Une opération entre un tableau et un nombre s'applique à **chaque case**. Entre deux tableaux, elle se fait case par case si leurs formes sont **compatibles** : identiques, ou l'une des dimensions vaut 1. Sinon, Python lève une `ValueError` .

GESTE	CODE	RETENIR
Créer	`np.array([..])`, `np.arange(5)`, `np.zeros((2, 3))`	`arange(5)` = 0 à 4
Indexer	`a[0]`, `a[-1]`, `m[ligne, colonne]`, `a[1:3]`	la fin d'une tranche est exclue
Filtrer	`a[a > 10]`, `a[(a > 5) & (a < 15)]`	`&` `|` `~` avec des parenthèses
Résumer	`.sum()` `.mean()` `.std()` `.min()` `.max()` `.argmax()`	`argmax` = la position du max
Par axe	`m.sum(axis=0)` / `m.sum(axis=1)`	`axis=0` écrase les lignes → un résultat par colonne
Reformer	`a.reshape(2, 3)`	le nombre total de cases doit rester le même

Pièges d'examen

1. `liste * 2` **répète** la liste ; `array * 2` **multiplie** chaque case.
2. `and` / `or` sur des tableaux → `ValueError` (« truth value is ambiguous »). Il faut `&` / `|`, avec des parenthèses autour de chaque condition.
3. Une tranche `a[1:3]` est une **vue**, pas une copie : la modifier modifie `a`. On écrit `a[1:3].copy()` (c'est la règle des étiquettes du Day 1, semaine 1).
4. `NaN` contamine les calculs : `np.mean([1, np.nan])` donne `nan`. Il faut `np.nanmean`.

CAS D'EXAMEN	RÉPONSE
`np.array([1, 2, 3]) + np.array([10, 20])`	`ValueError` : formes (3,) et (2,) incompatibles
`m = np.array([[1, 2], [3, 4]])` ; `m.sum(axis=0)` puis `m.sum(axis=1)`	`[4 6]` (par colonne) puis `[3 7]` (par ligne)
`a = np.array([1, 2, 3, 4])` ; `b = a[1:3]` ; `b[0] = 99` ; `print(a)`	`[ 1 99 3 4]` : `b` est une vue sur `a`

Lien ML : données, images, poids d'un réseau de neurones : tout est tableau. Entraîner un modèle, c'est enchaîner des opérations vectorisées sur ces tableaux.

2. Pandas : explorer un tableau de données DAY 2 · 2 MIN

Définition 2.1 *(DataFrame et Series)*

Un **DataFrame** est un tableau à colonnes nommées. Une colonne seule est une **Series** (un tableau NumPy avec un index). En ML : **une ligne = un exemple**, **une colonne = une feature**, et la colonne à prédire est la **cible**.

GESTE	CODE	RETENIR
Charger	`pd.read_csv("f.csv")`	le premier réflexe de tout projet
Inspecter	`.head()` `.shape` `.info()` `.describe()` `.dtypes`	`info()` montre les types et les valeurs manquantes
Colonne(s)	`df["a"]` → Series · `df[["a", "b"]]` → DataFrame	plusieurs colonnes = **double crochet**
Lignes	`df.iloc[0:2]` (position) · `df.loc[0:2]` (étiquette)	`iloc` exclut la fin, `loc` l'inclut
Filtrer	`df[df["ville"] == "Paris"]`	`==` demande, `=` écrase
Créer une colonne	`df["moy"] = (df["a"] + df["b"]) / 2`	feature engineering
Trier, compter	`df.sort_values("a", ascending=False)` · `df["v"].value_counts()`	`value_counts` = le `Counter` de Pandas
Agréger	`df.groupby("ville")["maths"].agg(["mean", "count"])`	grouper, choisir, calculer

Pièges d'examen

1. `df["a", "b"]` → `KeyError`. Il faut `df[["a", "b"]]`.
2. `df["ville"] = "Paris"` écrase toute la colonne, **sans erreur**.
3. Modifier une sélection filtrée (`df[df.x > 0]["y"] = 1`) ne modifie pas `df` et provoque un `SettingWithCopyWarning`. On écrit `df.loc[df["x"] > 0, "y"] = 1`.
4. Plusieurs conditions : `df[(df.a > 1) & (df.b < 5)]`, jamais `and`.

CAS D'EXAMEN	RÉPONSE
Type de `df["maths"]` et de `df[["maths"]]` ?	une `Series`, puis un `DataFrame` à une colonne
Index par défaut 0 à 4 : nombre de lignes de `df.iloc[0:2]` et de `df.loc[0:2]` ?	2 et 3
Moyenne de maths des élèves de Lyon ?	`df[df["ville"] == "Lyon"]["maths"].mean()`

3. Nettoyer des données sales

DAY 3 · 2 MIN

> **Principe 3.1** *(garbage in, garbage out)*
>
> La qualité d'un modèle ne dépasse jamais la qualité de ses données. Le nettoyage représente la majorité du temps d'un projet ML.

PROBLÈME	DÉTECTER	TRAITER
Doublons	`df.duplicated().sum()`	`df.drop_duplicates(subset=[...])`
Mauvais type	`df.dtypes` (`object` au lieu d'un nombre)	`pd.to_numeric(col, errors="coerce")` → `NaN`
Valeurs manquantes	`df.isna().sum()`	`dropna(subset=[...])` ou `fillna(df[c].median())`
Valeurs impossibles	`describe()` (min et max)	filtrer `df[(c >= 0) & (c <= 20)]` ou `clip(0, 20)`
Texte incohérent	`value_counts()`	`.str.strip().str.lower()`

> **Méthode 3.2** *(l'ordre de nettoyage)*
>
> 1. Inspecter (`info`, `describe`, `isna().sum()`). 2. Uniformiser le texte. 3. Convertir les types. 4. Supprimer les doublons. 5. Traiter les valeurs impossibles. 6. Traiter les manquants. 7. Revérifier, et noter combien de lignes ont été retirées.

Pièges d'examen

1. `NaN == NaN` vaut `False` : `df[df.x == np.nan]` est toujours vide. On utilise `df.x.isna()`.
2. `df = df.dropna(inplace=True)` met `None` dans `df` : avec `inplace=True`, la méthode renvoie `None`. On écrit `df = df.dropna()`.
3. Remplir par la **moyenne** quand il y a des valeurs extrêmes fausse le remplissage. On préfère la **médiane**.
4. `dropna()` sans `subset` supprime une ligne dès qu'**une** case est vide : on peut perdre la moitié du dataset.
5. **Fuite de données** : la valeur de remplissage se calcule sur le jeu d'entraînement seulement, jamais sur le jeu de test.

CAS D'EXAMEN	RÉPONSE
`pd.to_numeric(pd.Series(["12", "abc", ""]), errors="coerce")`	`[12.0, NaN, NaN]`
Salaires : 30k, 32k, 35k, 2 000k et une case vide. Remplir par quoi ?	la médiane (33,5k). La moyenne (524k) est tirée par la valeur extrême.

4. Visualiser pour comprendre DAY 4 · 2 MIN

> **Principe 4.1** *(une question, un graphique)*
>
> On choisit le graphique d'après la **question** posée, jamais d'après son apparence.

QUESTION	GRAPHIQUE	MATPLOTLIB
Comment une variable numérique est-elle répartie ?	histogramme	`plt.hist(x, bins=20)`
Où sont la médiane, la dispersion et les valeurs extrêmes ?	boîte à moustaches	`plt.boxplot(x)`
Quelle catégorie est la plus grande ?	barres	`plt.bar(noms, valeurs)`
Deux variables sont-elles liées ?	nuage de points	`plt.scatter(x, y)`
Comment une valeur évolue-t-elle dans le temps ?	courbe	`plt.plot(t, y)`

Méthode 4.2 *(un graphique lisible)*

`plt.figure()` → tracer → `plt.title()`, `plt.xlabel()`, `plt.ylabel()` (avec les unités), `plt.legend()` s'il y a plusieurs séries → `plt.savefig("f.png")` **avant** `plt.show()`, sinon l'image enregistrée est vide.

Pièges d'examen

1. **Barres et histogramme** : les barres comparent des **catégories** ; l'histogramme découpe une variable **continue** en intervalles.
2. Le choix de `bins` change la forme : trop peu d'intervalles cachent tout, trop d'intervalles donnent du bruit.
3. Un axe vertical qui ne part pas de 0 exagère les écarts entre les barres.
4. Des points alignés montrent une **corrélation, pas une causalité**.

Lien ML : l'histogramme révèle les valeurs aberrantes et les classes déséquilibrées ; le nuage de points dit si une feature aide à prédire la cible ; la courbe d'apprentissage montre si l'erreur baisse à chaque époque.

5. Statistiques pour le ML DAY 5 · 2 MIN

MESURE	DÉFINITION	PANDAS	RETENIR
Moyenne μ	somme ÷ effectif	`.mean()`	sensible aux valeurs extrêmes
Médiane	la valeur du milieu, une fois les valeurs triées	`.median()`	**robuste** aux valeurs extrêmes
Mode	la valeur la plus fréquente	`.mode()`	utile pour les catégories
Écart-type σ	l'écart typique à la moyenne (racine de la variance)	`.std()`	dans la même unité que les données ; 0 si toutes les valeurs sont égales
Quartiles	Q1 = 25 %, Q2 = médiane, Q3 = 75 %	`.quantile(0.25)`	IQR = Q3 − Q1
Corrélation r	force du lien **linéaire**, entre −1 et 1	`df["a"].corr(df["b"])`	proche de 1 : monte ensemble ; proche de −1 : sens inverse ; 0 : pas de lien linéaire

Proposition 5.1 *(règle des valeurs aberrantes)*

Une valeur est considérée comme aberrante si elle est inférieure à Q1 − 1,5 × IQR ou supérieure à Q3 + 1,5 × IQR. C'est la règle utilisée par la boîte à moustaches.

Proposition 5.2 *(loi normale : 68 – 95 – 99,7)*

Pour des données en cloche, environ 68 % des valeurs sont à moins d'un écart-type de la moyenne, 95 % à moins de deux, et 99,7 % à moins de trois.

Définition 5.3 *(mise à l'échelle)*

Standardisation : z = (x − μ) / σ, ce qui donne une moyenne de 0 et un écart-type de 1. **Normalisation min-max** : (x − min) / (max − min), ce qui ramène les valeurs entre 0 et 1. Beaucoup de modèles l'exigent pour qu'une feature en milliers (un salaire) n'écrase pas une feature en unités (un âge).

Pièges d'examen

1. **Corrélation n'est pas causalité** : les ventes de glaces et les noyades montent ensemble, et c'est la chaleur qui explique les deux.
2. r = 0 ne veut pas dire « aucun lien » : $y = x^2$ sur des x symétriques donne r = 0, alors que le lien est parfait (mais pas linéaire). Il faut toujours tracer le nuage de points.
3. Une moyenne seule ne suffit pas : deux classes de moyenne 10 peuvent avoir des écarts-types de 0,5 et de 6.

CAS D'EXAMEN	RÉPONSE
Moyenne et médiane de [1, 2, 3, 4, 100] ?	22 et 3 : la médiane résiste à la valeur extrême
Q1 = 8, Q3 = 14. La valeur 25 est-elle aberrante ?	Oui : IQR = 6, la limite haute vaut 14 + 9 = 23, et 25 > 23
$\mu = 10$, $\sigma = 2$. Que vaut z pour x = 14 ?	$z = (14 - 10) / 2 = 2$: deux écarts-types au-dessus de la moyenne